PRD 01 — VolatilityLab

Advanced Options Pricing, Volatility Forecasting & Quantitative Risk Platform

Version 1.0

October 30, 2026 · Portfolio / Research Project

1. Executive summary

VolatilityLab is a full-stack quantitative finance research platform that combines traditional derivatives pricing models, stochastic simulations, machine learning, and portfolio risk analytics.

The platform will allow users to retrieve historical and current market data, price European and American options, calculate option Greeks, calibrate volatility models, forecast realized volatility using ML, and evaluate hedging strategies through historical backtesting.

Primary objective: Demonstrate professional-level quantitative research, numerical finance, machine learning, and software engineering through a reproducible, well-tested, interactive application.

Target audiences

- Quantitative researchers and traders
- Derivatives analysts and risk professionals
- ML researchers working on financial time series
- Technical recruiters and hiring managers evaluating your portfolio

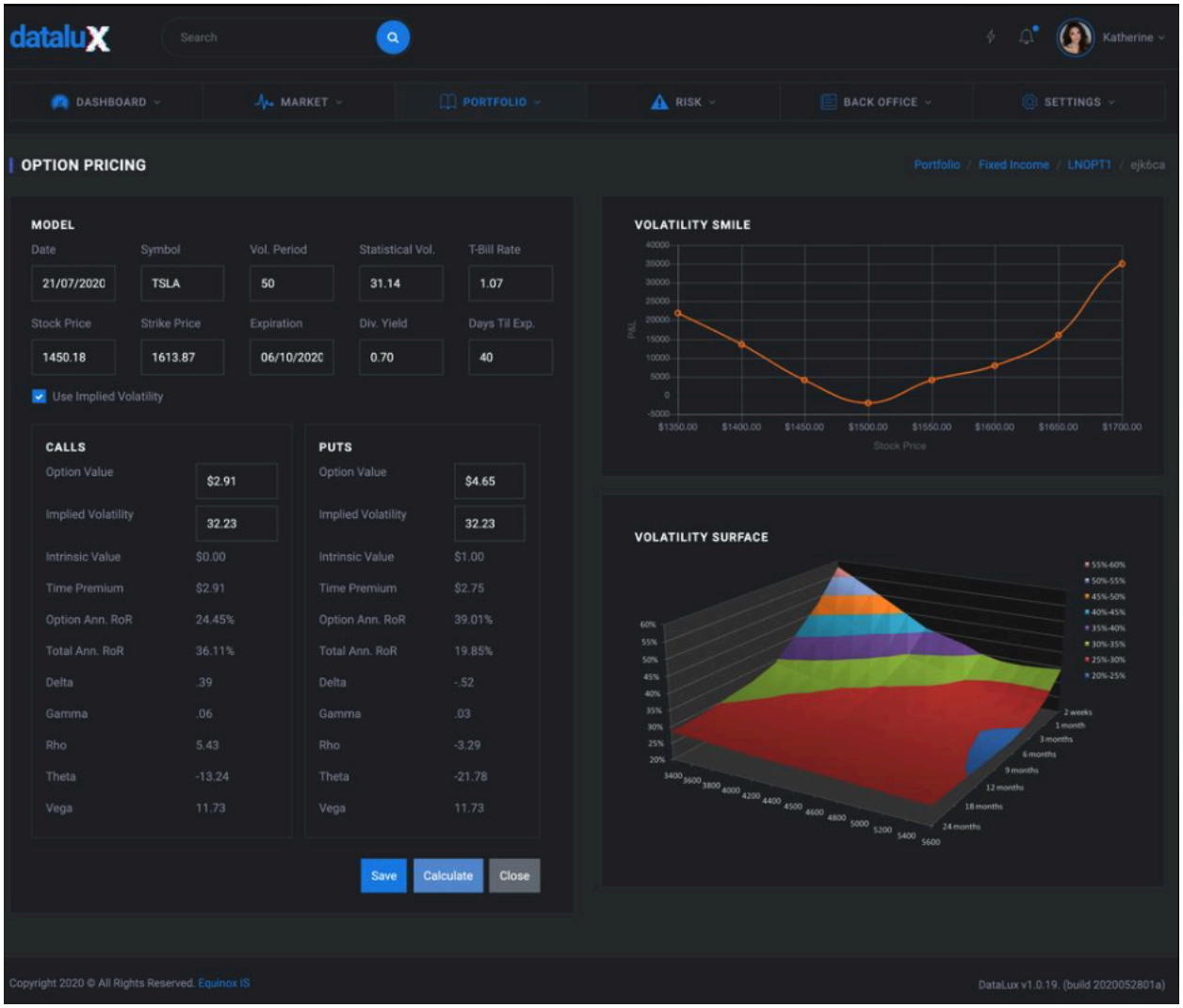
Portfolio differentiation

Unlike a typical Black–Scholes calculator, VolatilityLab will integrate pricing theory with real-market observations, model calibration, ML-based forecasts, uncertainty estimation, and rigorous out-of-sample evaluation.

2. Product vision and scope

Target user experience

DataLux Reviews in 2025
sourceforge.net
Visual design reference. The actual interface will be implemented using React and Plotly.



Core workflow

1

Select underlying and option contract

2

Load market prices and volatility data

3

Compare pricing models

4

Inspect Greeks and volatility surfaces

5

Run ML forecasts and hedge simulations

6

Export reproducible research report

In scope

Capability	MVP	Advanced
Black–Scholes–Merton	Yes	
Binomial tree pricing	Yes	
Monte Carlo pricing	Yes	
Greeks and sensitivities	Yes	
Implied volatility solver	Yes	
Volatility surface visualization	Yes	
Heston stochastic volatility		Yes
GARCH and XGBoost forecasting		Yes
LSTM volatility forecasting		Yes

Delta-hedging backtesting	Yes
Scenario risk and VaR/ES	Yes
Automated research reports	Yes

Out of scope

Live brokerage execution, real-money trading, automated order placement, market-making infrastructure, and claims of guaranteed trading profitability.

The initial product is a research and analytics system, not an investment recommendation engine.

3. Functional requirements

FR-01 — Market data ingestion

The platform must ingest and normalize underlying price histories, options chains, contract metadata, interest rates, and dividend assumptions.

Requirements:

- Support equities and ETFs, beginning with SPY, QQQ, and NVDA.
- Retrieve available option chains with bid, ask, strike, expiration, type, and open interest.
- Preserve observation timestamps and data-source provenance.
- Store raw and cleaned datasets separately.
- Detect missing values, stale quotes, crossed markets, and invalid contracts.
- Permit CSV/Parquet uploads when historical options data is unavailable through an API.
- Cache retrieved datasets and support reproducible historical snapshots.

Acceptance criteria: A user can load a supported option chain, view the quality filters applied, and reproduce the same analysis from a saved snapshot.

FR-02 — Black–Scholes–Merton pricing engine

Implement European call and put pricing with continuous dividend yield.

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2)$$

$$P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1)$$

Where S is the spot price, K the strike, T time to expiration, r the continuously compounded risk-free rate, q the continuous dividend yield, and σ volatility.

Requirements:

- Support calls and puts.
- Implement analytical Delta, Gamma, Vega, Theta, and Rho.
- Handle expiration, near-zero volatility, and deep ITM/OTM cases.
- Support vectorized calculations using NumPy.
- Validate against trusted reference implementations and put-call parity.

Acceptance criteria: Standard test cases agree with independently validated reference results within documented numerical tolerances.

FR-03 — Binomial option pricing

Implement the Cox–Ross–Rubinstein model.

Requirements:

- European and American call/put support.
- Configurable number of tree steps.
- Early-exercise logic.
- Convergence analysis against analytical European prices.
- Greeks estimated through appropriate numerical methods.

Acceptance criteria: European binomial prices converge toward Black–Scholes prices as resolution increases; American option prices respect early-exercise constraints and benchmark cases.

FR-04 — Monte Carlo simulation

Implement risk-neutral geometric Brownian motion:

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t$$

Requirements:

- Configurable paths, time steps, and random seed.

- European option payoff estimation.
- Standard errors and confidence intervals.
- Antithetic variates and optional control variates.
- Convergence diagnostics.
- Vectorized and memory-conscious simulation.

Acceptance criteria: Monte Carlo estimates are statistically consistent with Black–Scholes benchmark prices and confidence interval behavior is verified across repeated experiments.

FR-05 — Implied volatility and surface construction

Requirements:

- Implement a robust implied-volatility solver using Brent's method.
- Reject quotes that violate no-arbitrage price bounds.
- Calculate bid-IV, ask-IV, and mid-IV where valid.
- Plot implied volatility by strike or log-moneyness and expiration.
- Provide smile, skew, and term-structure views.
- Clearly distinguish observed points from interpolated surfaces.
- Flag sparse data and extrapolation regions.

Acceptance criteria: Repricing an option using its solved implied volatility reproduces the input price within a configurable tolerance.

FR-06 — Stochastic volatility model

Implement the Heston model:

$$dS_t = (r - q)S_t dt + \sqrt{v_t} S_t dW_t^S$$

$$dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t} dW_t^v$$

with correlation:

\[dW_t^S dW_t^v=\rho\,dt \]

Requirements:

- Heston characteristic-function pricing for European options.
- Parameter calibration to market option quotes.
- Constraints on variance parameters and correlation.
- Calibration error metrics.
- Parameter sensitivity analysis.

- Numerical stability and convergence tests.

This is an advanced research module and must not block MVP completion.

FR-07 — Machine learning volatility forecasting

The platform must compare traditional volatility models with ML methods.

Prediction target: Future realized variance or volatility over a fixed horizon, such as five or twenty trading days.

Models:

- Rolling historical volatility baseline
- EWMA
- GARCH(1,1)
- XGBoost
- LSTM or Temporal Convolutional Network

Features:

- Lagged returns
- Realized volatility
- Volatility-of-volatility
- Trading volume
- Option-implied volatility, where historically available
- Term-structure features
- Market stress indicators

Evaluation: MAE, RMSE, QLIKE for strictly positive variance forecasts, and forecast calibration diagnostics.

Use chronological training, validation, and testing with appropriate gaps for overlapping targets. Hyperparameters must be selected without accessing the final test period.

FR-08 — Delta-hedging simulation

Requirements:

- Simulate a delta-hedged option position over its lifetime.
- Support daily and configurable rebalance intervals.
- Include underlying trading costs, financing, and option bid-ask costs where applicable.
- Track option P&L, hedge P&L, financing, and transaction costs.
- Compare hedge performance under alternative volatility estimates.
- Report realized hedging errors and their distribution.

Acceptance criteria: A complete simulated hedge can be reconciled into its cash flows and terminal P&L.

FR-09 — Risk analytics

Provide:

- Portfolio Greeks
- Volatility shock scenarios
- Underlying price shock scenarios
- Historical simulation VaR
- Expected shortfall
- P&L distribution charts

Risk calculations must identify their confidence level, time horizon, assumptions, and limitations.

FR-10 — Research reporting

Generate downloadable reports containing:

- Dataset description and observation dates
- Pricing-model assumptions
- Calibration parameters
- Model comparison tables
- Forecast evaluation
- Hedging simulation results
- Figures and methodology
- Reproducibility metadata

The reports should be generated from saved experiment outputs rather than manually copied dashboard numbers.

4. System architecture

React + TypeScript Frontend

Plotly · Tailwind CSS · TanStack Query

FastAPI Application Layer

REST API · Pydantic · Authentication optional

Pricing Engine

Volatility Models

Risk & Backtesting

ML Training Pipeline

PostgreSQL

Metadata, contracts, experiments

Parquet Storage

Historical data and model outputs

External Market Data Providers

Options · Underlyings · Rates · Corporate actions

Technology stack

Layer	Technology
Backend	Python 3.12+, FastAPI
Numerical computing	NumPy, SciPy
Data engineering	pandas, Polars, PyArrow
ML	scikit-learn, XGBoost, PyTorch
Econometrics	statsmodels, arch
Database	PostgreSQL, SQLAlchemy
Experiment tracking	MLflow

Frontend	React, TypeScript, Plotly
Testing	pytest, Hypothesis
CI/CD	GitHub Actions
Deployment	Docker, Docker Compose

5. Data model

The system should use the following primary entities.

Entity	Important fields
`underlyings`	id, symbol, asset_type, exchange
`option_contracts`	id, underlying_id, expiration, strike, option_type, exercise_style, multiplier
`market_snapshots`	id, provider, observed_at, ingested_at
`option_quotes`	contract_id, snapshot_id, bid, ask, volume, open_interest
`underlying_prices`	underlying_id, timestamp, adjusted_close, raw_close, volume

`rate_curves`	as_of_date, tenor, rate, source
`pricing_runs`	id, model_name, parameters, contract_id, model_price, runtime_ms
`ml_experiments`	id, target, horizon, train_period, test_period, metrics, artifact_uri
`hedge_backtests`	id, strategy_config, period, total_pnl, cost_pnl, artifact_uri

Every experiment must preserve code version, random seed, data snapshot identifier, configuration, and model artifacts.

6. API specification

Endpoint	Method	Purpose
`/api/v1/underlyings`	GET	List supported assets
`/api/v1/options/chain`	GET	Retrieve normalized options chain
`/api/v1/pricing/black-scholes`	POST	Calculate theoretical price and Greeks
`/api/v1/pricing/binomial`	POST	Calculate tree-based option price
`/api/v1/pricing/monte-carlo`	POST	Execute Monte Carlo valuation

`/api/v1/volatility/implied`	POST	Calculate implied volatility
`/api/v1/volatility/surface`	GET	Retrieve saved surface
`/api/v1/experiments`	POST	Launch an ML experiment
`/api/v1/experiments/{id}`	GET	Retrieve experiment status and results
`/api/v1/backtests`	POST	Launch hedge simulation
`/api/v1/reports/{id}`	GET	Retrieve generated report

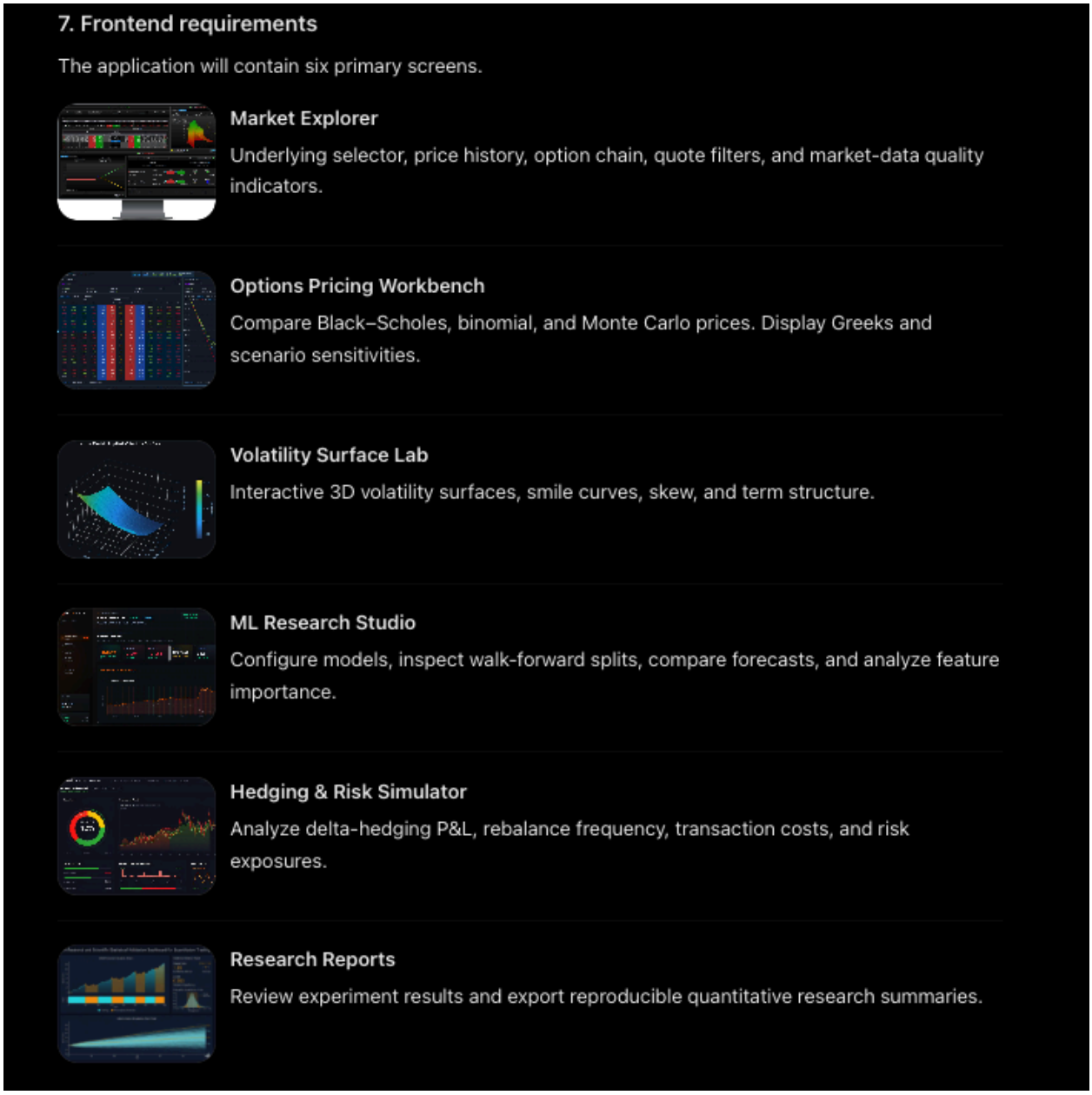
Computationally expensive requests should be executed asynchronously through a job queue rather than blocking API workers.

Example pricing request

```
None
  - {
  -   "option_type": "call",
  -   "spot": 500.0,
  -   "strike": 510.0,
  -   "time_to_expiry": 0.25,
  -   "risk_free_rate": 0.04,
  -   "dividend_yield": 0.01,
  -   "volatility": 0.22
  }
```

The API response should include the theoretical price, Greeks, model identifier, assumptions, and numerical warnings when applicable.

7. Frontend requirements



The application will contain six primary screens.

SPACE EXPLORATION TECHNOLOGIES Stock price | NASDAQ:SPCX Quote and Chart

Market Explorer

Underlying selector, price history, option chain, quote filters, and market-data quality indicators.

Multi-Asset Trading Platform — TraderEvolution

Options Pricing Workbench

Compare Black–Scholes, binomial, and Monte Carlo prices. Display Greeks and scenario sensitivities.

Heston Stochastic Volatility Model: Pricing, Calibration & Monte Carlo | SimTool

Volatility Surface Lab

Interactive 3D volatility surfaces, smile curves, skew, and term structure.

MASI Hybrid Forecasting

ML Research Studio

Configure models, inspect walk-forward splits, compare forecasts, and analyze feature importance.

Drawdown Monitor – Real-Time Funded Account Risk Tracking | RockstarTrader

Hedging & Risk Simulator

Analyze delta-hedging P&L, rebalance frequency, transaction costs, and risk exposures.

AlgoriQ - ML-Based Quant Trading Platform

Research Reports

Review experiment results and export reproducible quantitative research summaries.

8. ML research methodology

The ML component must be designed as a genuine empirical study rather than an arbitrary forecasting demonstration.

Hypotheses

H1: ML-based volatility forecasts outperform historical-volatility and EWMA baselines on at least one prespecified out-of-sample forecasting metric.

H2: Improvements in statistical volatility forecasts do not necessarily translate into improved option-hedging performance after costs.

H3: Forecast performance differs across low- and high-volatility market regimes.

These are hypotheses to test, not assumed outcomes.

Experimental protocol

1. Define forecast horizons before training.
2. Construct point-in-time features using only information available at prediction time.
3. Establish chronological training, validation, and untouched test periods.
4. Tune models using rolling or expanding-window validation.
5. Fit preprocessing exclusively on the corresponding training window.
6. Compare all models on identical test timestamps.
7. Estimate uncertainty using a dependence-aware bootstrap or suitable forecast comparison tests.
8. Publish failures, regime sensitivity, and limitations alongside successful results.

Important financial distinction: A realized-volatility forecast is not automatically a risk-neutral implied-volatility forecast. The platform must explicitly document how any forecast is used in pricing or hedging experiments, including the volatility risk premium and calibration assumptions.

9. Non-functional requirements

Requirement	Target
Standard Black–Scholes calculation	Under 100 ms at the API level under normal local conditions
Standard dashboard navigation	Responsive without blocking long-running jobs
Reproducibility	Identical seeded experiments reproduce results within numerical tolerances
Test coverage	At least 85% on core pricing and risk modules

Numerical accuracy	Explicit model-specific tolerances and convergence benchmarks
Security	Validated inputs, environment-based secrets, no exposed provider keys
Observability	Structured logs, experiment IDs, job status, error tracing
Portability	Runs locally using documented Docker Compose setup

Performance targets are engineering goals to benchmark, not pre-established results.

10. Testing and validation

The testing strategy must go beyond basic unit tests.

Numerical tests: Put-call parity, no-arbitrage bounds, monotonicity with strike, known Greeks, binomial convergence, Monte Carlo confidence intervals, and implied-volatility inversion.

Property-based tests: Generate valid randomized option parameters and verify mathematical invariants.

Integration tests: Confirm market-data ingestion, database persistence, API response validation, and saved experiment retrieval.

ML tests: Detect feature leakage, invalid time splits, nondeterministic preprocessing, and differences between training and inference transformations.

End-to-end tests: Load a contract, price it, visualize its volatility, run an experiment, and export a report.

11. Repository architecture

```
None
  - volatilitylab/
  - ├── README.md
```

```
- ├── LICENSE
- ├── pyproject.toml
- ├── docker-compose.yml
- ├── .env.example
- ├── .github/
- │   └── workflows/
- │       ├── tests.yml
- │       └── lint.yml
- ├── backend/
- │   └── app/
- │       ├── main.py
- │       ├── api/
- │       ├── schemas/
- │       ├── services/
- │       └── database/
- ├── src/
- │   └── volatilitylab/
- │       ├── data/
- │       ├── pricing/
- │       │   ├── black_scholes.py
- │       │   ├── binomial.py
- │       │   ├── monte_carlo.py
- │       │   └── heston.py
- │       ├── volatility/
- │       │   ├── implied_vol.py
- │       │   ├── surface.py
- │       │   └── forecasting.py
- │       ├── ml/
- │       │   ├── features.py
- │       │   ├── validation.py
- │       │   ├── models.py
- │       │   └── evaluation.py
- │       ├── risk/
- │       │   ├── greeks.py
- │       │   ├── var.py
- │       │   └── hedging.py
```

```
-   │       └── backtesting/
-   ├── frontend/
-   │   ├── src/
-   │   └── package.json
-   ├── tests/
-   │   ├── unit/
-   │   ├── integration/
-   │   └── numerical/
-   ├── notebooks/
-   ├── experiments/
-   ├── configs/
-   ├── docs/
-   │   ├── architecture.md
-   │   ├── mathematical_models.md
-   │   ├── data_dictionary.md
-   │   └── methodology.md
    └── scripts/
```

12. Development roadmap

Implementation checklist

0 / 24 tasks

1. Foundation & pricing

Week 1

Set up monorepo, CI, Docker, and API skeleton

Implement Black–Scholes and analytical Greeks

Implement binomial pricing and numerical tests

Create initial pricing workbench

2. Simulation & market data

Week 2

Implement Monte Carlo engine and confidence intervals

Build market-data ingestion and validation

Persist market snapshots and option contracts

Add pricing-model comparison views

3. Volatility analytics

Week 3

Build implied-volatility root solver

Construct smile and surface visualizations

Implement quote-quality filters

Document no-arbitrage and interpolation assumptions

4. MVP integration

Week 4

Connect complete frontend and backend

Add integration and end-to-end tests

Benchmark pricing accuracy and latency

Publish README, demo, and reproducible sample dataset

5. Advanced research

Weeks 5–7

Implement GARCH, XGBoost, and LSTM experiments

Build walk-forward evaluation and MLflow tracking

Implement Heston calibration

Add delta-hedging simulations and risk analytics

6. Research publication

Week 8

Perform final untouched out-of-sample evaluation

Produce figures and statistical comparisons

Write technical research report

Record demo and publish documented results

13. MVP definition of done

The MVP is complete when a reviewer can clone the repository, start the application locally, and perform the following workflow without manual code changes:

1. Load a documented option contract or sample options chain.
2. Calculate Black–Scholes, binomial, and Monte Carlo prices.
3. Inspect Greeks and model discrepancies.
4. Calculate implied volatility from a valid quote.
5. Visualize a volatility smile or surface.
6. Reproduce a saved analysis using fixed input data.
7. Run the test suite successfully.

The advanced research release additionally requires documented ML experiments, a valid out-of-sample evaluation, a hedging backtest, and a technical report.

14. GitHub portfolio presentation

The repository's main README should communicate its value within the first thirty seconds.

volatilitylab

Quantitative Options Pricing, Volatility Forecasting & Risk Analytics

An open-source research platform combining numerical derivatives pricing, stochastic simulation, machine learning, and empirical hedging analysis.

Python

FastAPI

React

PyTorch

XGBoost

NumPy

SciPy

PostgreSQL

README sections: Demo · Research Questions · Architecture · Mathematical Models · Experiments · Results · Reproduction · Limitations

The most important portfolio deliverable is not the interface alone. It is a technical research report demonstrating that you can derive models, implement them correctly, design valid experiments, interpret results, and communicate limitations.

For this project, I recommend publishing a report titled:

Comparative Evaluation of Stochastic Option Pricing Models and Machine Learning Volatility Forecasts Under Realistic Hedging Costs.

That would make VolatilityLab relevant to quant research and trading interviews while also demonstrating the ML and software engineering skills you're developing in your master's program.

-